

AWS IAM

Professional Reference

Solution Architect Professional Level

Covers: Policy Evaluation · Roles & STS · Cross-Account · Permission Boundaries · Session Policies · SCPs · Resource Policies · Conditions · ABAC · EKS Pod Identity · IRSA · IAM Terminology · Anti-Patterns

Tharun | AWS SAP Revision Guide | 2025

Table of Contents

CH 01 IAM Fundamentals Recap

- Users, Groups, Roles, Policies overview

CH 02 Policy Evaluation Logic

- Explicit DENY > ALLOW > Default DENY
- Evaluation order: Identity → Resource → Boundary → SCP → Session
- Full evaluation path walkthrough

CH 03 IAM Roles — Deep Dive

- Trust Policy vs Permission Policy
- Role assumption flow
- Real-world failure patterns

CH 04 STS — Security Token Service

- AssumeRole · AssumeRoleWithWebIdentity · AssumeRoleWithSAML
- Temporary credential lifecycle

CH 05 Cross-Account Access

- Flow & trust policy setup
- External ID & Confused Deputy Problem

CH 06 Permission Boundaries

- Definition, analogy, examples
- Delegating IAM safely, preventing privilege escalation

CH 07 Session Policies

- What, where, when
- Safer automation · Multi-tenant · Debugging
- Session policy vs Permission Boundary

CH 08 SCPs — Service Control Policies

- Org-level guardrails
- SCP + IAM interaction

CH 09 Resource-Based Policies

- IAM Policy vs Resource Policy
- S3, Lambda, SNS, SQS examples

CH 10 IAM Conditions

- Key operators: IpAddress, MFA, Region, PrincipalTag
- ABAC — Attribute-Based Access Control

CH 11 EKS Pod Identity & IRSA

- Pod Identity (modern way) — step-by-step
- IRSA (OIDC) — step-by-step
- Pod Identity vs IRSA comparison
- Pod compromise isolation

CH 12 IAM Terminology

- Identity · Entity · Principal · Resource
- Role vs Principal distinction
- Real-world EKS scenario tying all terms

CH 13 Anti-Patterns & Best Practices

- Top 10 IAM mistakes
- AccessDenied debug checklist

Core IAM Objects

Object	What it is	Credentials	Use case
IAM User	Long-term identity for a human or service	Password + Access Keys	Human console login, legacy SDK access
IAM Group	Collection of users; policies attached to group	None (group can't log in)	Bulk permission management
IAM Role	Assumable identity with no long-term creds	Temporary (STS tokens)	EC2, Lambda, cross-account, EKS pods
IAM Policy	JSON permission document	—	Attached to users/groups /roles/resources

Policy Types

- ◆ **Identity-based** — Attached to a user, group, or role. Defines WHAT they can do.
- ◆ **Resource-based** — Attached to the resource (S3 bucket, SQS queue). Defines WHO can access it. Supports cross-account.
- ◆ **Permission Boundary** — Inline policy that sets the MAXIMUM permissions a user/role can ever have. Does NOT grant — only restricts.
- ◆ **SCP (Service Control Policy)** — AWS Organizations-level guardrail. Filters what actions are allowed in an account — does not grant permissions.
- ◆ **Session Policy** — Inline policy passed during STS AssumeRole call. Reduces permissions for that specific session only.
- ◆ **ACLs** — Legacy; used by S3 and VPC. Less granular than resource policies.

Policy Evaluation Logic

The most important concept — where most engineers fail

The Golden Rule

AWS IAM is NOT just allow/deny. It is a layered evaluation engine. Companies get breached not because IAM is broken, but because evaluation logic + misconfigurations + trust relationships are misunderstood.

Final Decision Formula

Explicit DENY > ALLOW > Default DENY

■ Think like a secure building: any guard saying NO means you're OUT. Otherwise you need at least one YES to get IN.

Evaluation Order (All 5 Layers)

- 1 Explicit DENY** — Any policy at any layer that explicitly denies — game over. Always checked first.
- 2 SCPs (Org level)** — If the account is in an AWS Organization, the SCP must allow the action. SCPs never grant — they only filter.
- 3 Resource-based policies** — For services like S3, Lambda, SQS: the resource policy is checked. Cross-account access requires BOTH identity policy AND resource policy to allow.
- 4 Identity-based policies** — IAM User/Group/Role policies attached to the calling identity.
- 5 Permission Boundary** — If a boundary is attached to the role/user, the effective permission must be in BOTH the identity policy AND the boundary.

Full Evaluation Formula

Final Permissions = Role/User Policy $\cap$ Permission Boundary $\cap$ Session Policy $\cap$ SCP

■ $\cap$ means intersection — all layers must allow the action for it to succeed.

Real-World Failure Example

Scenario: You attach AdministratorAccess to a role, but EC2 is still blocked.

Layer	What's set	Result
IAM Role Policy	AdministratorAccess (allow *)	✓ Allows EC2
SCP (Org)	Deny ec2:*	✗ Explicit DENY wins
Final	—	EC2 BLOCKED

■ **Key insight:** Many engineers debug IAM wrongly because they **ONLY** check the attached policy. You must check **ALL** layers: SCP → Boundary → Session → Resource policy.

AccessDenied Full Debug Path

When you see AccessDenied on a role that 'should have access', walk this path:

- ◆ **Step 1:** Is there an explicit Deny in any identity-based policy?
- ◆ **Step 2:** Is there an SCP in the AWS Organization that blocks the action?
- ◆ **Step 3:** Does the resource-based policy (S3, Lambda, etc.) deny or not allow?
- ◆ **Step 4:** Does the Permission Boundary restrict this action?
- ◆ **Step 5:** Does a Session Policy restrict this specific session?
- ◆ **Step 6:** Is the trust policy correct? (WHO can assume the role)
- ◆ **Step 7:** Is the action in the right region / resource ARN?

Temporary identities — the backbone of AWS security

What Makes a Role Different from a User

- ✓ **No password** — Roles cannot have console passwords
- ✓ **No long-term access keys** — Credentials issued via STS are temporary
- ✓ **Assumable by any trusted principal** — EC2, Lambda, another account, an EKS pod
- ✓ **Automatic rotation** — Temp creds expire; no manual key rotation needed

Trust Policy vs Permission Policy

Policy Type	Purpose	Question it answers	Where it lives
Trust Policy	Defines WHO can assume the role	"Can this entity assume me?"	Role's trust relationship tab
Permission Policy	Defines WHAT the role can do	"What can I do once assumed?"	Permissions tab / managed policy

Most Common Mistake

- People attach a permission policy but forget the trust policy. Result: AccessDenied even though the policy looks correct. ALWAYS check BOTH policies.

Trust Policy Example — EC2 Assuming a Role

Trust Policy (who can assume)

```
{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Principal": {  
  "Service": "ec2.amazonaws.com" }, "Action": "sts:AssumeRole" }] }
```

Role Assumption Flow

- ◆ 1. Entity (user/service/pod) calls sts:AssumeRole
- ◆ 2. AWS checks the role's TRUST POLICY — is this entity allowed to assume?
- ◆ 3. If allowed, STS issues temporary credentials: AccessKeyId + SecretAccessKey + SessionToken

- ◆ 4. Entity uses these creds to make AWS API calls
- ◆ 5. Every API call is evaluated against the role's PERMISSION POLICY (+ boundary + SCP + session)
- ◆ 6. Credentials expire automatically (default 1 hour, max 12 hours for roles)

STS — Security Token Service

The engine behind all temporary credentials in AWS

What STS Generates

Every STS call returns a set of temporary credentials valid for a configurable duration. Think of STS as a temporary visitor pass generator for the AWS cloud.

Credential	Description
AccessKeyId	Unique identifier for the temporary session
SecretAccessKey	Secret used to sign API requests
SessionToken	MUST be included in every request; proves creds are temporary
Expiration	Timestamp when the creds expire (default 1h, max 12h for assumed roles)

Key STS APIs

API	Use case	Principal type
AssumeRole	Cross-account or intra-account role assumption	IAM user, role, service
AssumeRoleWithSAML	Enterprise SSO via SAML 2.0 IdP (Active Directory, Okta)	Federated SAML user
AssumeRoleWithWebIdentity	OAuth 2.0 / OIDC — used by EKS IRSA, mobile apps	OIDC token holder
GetSessionToken	Get temp creds for an IAM user (enforces MFA)	IAM user
GetFederationToken	Issue creds to a federated user (not a role)	IAM user or root

EKS IRSA — AssumeRoleWithWebIdentity Flow

This is the most architecturally significant STS API for Kubernetes workloads:

- ◆ 1. Kubernetes issues a signed OIDC JWT token to the pod (projected service account token)
- ◆ 2. Pod's AWS SDK sends this JWT to STS via AssumeRoleWithWebIdentity
- ◆ 3. STS validates the JWT signature against the cluster's OIDC provider endpoint
- ◆ 4. STS checks the IAM role's trust policy — does it allow this OIDC provider + service account?
- ◆ 5. If valid, STS returns temporary credentials to the pod

- ◆ 6. Pod uses creds to access S3, DynamoDB, etc. — no static keys anywhere

Why Cross-Account Access Matters

Enterprise AWS setups use multiple accounts for blast-radius isolation. A typical org has: Prod account, Dev/Staging account, Security/Audit account, Shared Services account. You need controlled, auditable access between them without distributing long-term credentials.

Cross-Account Role Assumption Flow

- ◆ 1. Account A creates an IAM Role with a Permission Policy (WHAT it can do).
- ◆ 2. Account A's Trust Policy explicitly names Account B (or a specific role in B) as a trusted principal.
- ◆ 3. A user/role in Account B calls `sts:AssumeRole` with the ARN of Account A's role.
- ◆ 4. STS issues temporary credentials scoped to Account A's role.
- ◆ 5. The caller now acts in Account A with exactly the permissions defined in that role.

Cross-Account Trust Policy

```
# Account A trust policy – trusting entire Account B: { "Effect": "Allow",  
"Principal": { "AWS": "arn:aws:iam::ACCOUNT_B_ID:root" }, "Action": "sts:AssumeRole"  
} # Tighter – trust only a specific role in Account B: { "Effect": "Allow",  
"Principal": { "AWS": "arn:aws:iam::ACCOUNT_B_ID:role/AnalyticsRole" }, "Action":  
"sts:AssumeRole" }
```

External ID — The Confused Deputy Problem

When a third-party SaaS (e.g., a monitoring vendor) needs access to your AWS account, you give them a role ARN. The problem: the vendor's AWS account is shared by many customers. An attacker could trick the vendor into using YOUR role ARN with THEIR account. External ID prevents this.

Scenario	Without External ID	With External ID
Third-party SaaS access	Attacker can supply your role ARN to the vendor and gain access	Vendor must supply your unique External ID — attacker doesn't know it

Trust Policy with External ID

```
{ "Effect": "Allow", "Principal": { "AWS": "arn:aws:iam::VENDOR_ACCOUNT:root" },  
"Action": "sts:AssumeRole", "Condition": { "StringEquals": { "sts:ExternalId":  
"unique-secret-value-you-share-with-vendor" } } }
```

■ Real-world failure: Forgot to allow correct account ID, or missing External ID for third-party access. Always specify the narrowest principal possible.

Permission Boundaries

Static guardrails — maximum permission ceiling for users and roles

Definition

A permission boundary is an IAM managed policy attached to a user or role that sets the **MAXIMUM** permissions that entity can ever have — regardless of what identity-based policies allow.

■ ■ Analogy: IAM Policy = what you **ASK FOR**. Permission Boundary = what your **BOSS** allows at most. Even if you request more, the boss says 'You cannot go beyond this.'

How It Works — Intersection Logic

Effective Permission = Identity Policy $\cap$ Permission Boundary

The intersection means **BOTH** must allow the action. If the identity policy allows `ec2:*` but the boundary only allows `s3:*`, then EC2 is denied.

Example

Permission Boundary in Action

```
# IAM Role Policy: { "Effect": "Allow", "Action": "ec2:*", "Resource": "*" } #  
Permission Boundary attached to role: { "Effect": "Allow", "Action": "s3:*",  
"Resource": "*" } # Result: # ec2:* — DENIED (not in boundary) # s3:* — ALLOWED (in  
both policy and boundary)
```

Critical Distinction

Belief	Reality
'Boundary grants permission'	WRONG — Boundary only RESTRICTS . It never grants. You need BOTH boundary AND identity policy to allow.
'Boundary applies org-wide'	WRONG — Boundary is per-user or per-role. SCPs are org-wide.
'Boundary replaces identity policy'	WRONG — Both must allow (intersection). Boundary doesn't replace, it caps.

Real-World Use Case: Delegating IAM Creation Safely

You want developers to create their own IAM roles (self-service) without being able to grant themselves admin access. Solution: allow `iam:CreateRole` + `iam:AttachRolePolicy`, but require that every role they create has a specific boundary attached.

Forcing Boundary on Delegated Role Creation

```
# Require devs to attach this boundary when creating roles: { "Effect": "Allow",  
"Action": ["iam:CreateRole", "iam:AttachRolePolicy"], "Resource": "*", "Condition": {  
"StringEquals": { "iam:PermissionsBoundary":  
"arn:aws:iam::ACCOUNT:policy/DevBoundary" } } }
```

■ This pattern prevents privilege escalation: dev cannot create a role more powerful than `DevBoundary`.

Temporary runtime restrictions — passed at role assumption time

Definition

A session policy is an inline policy you pass at the moment you call `sts:AssumeRole` (or `AssumeRoleWithWebIdentity` / `AssumeRoleWithSAML`) to FURTHER RESTRICT what that role can do during that specific session. It does NOT persist — it exists only for the duration of those temporary credentials.

■ ■ Analogy: Role = full access badge for the building. Session Policy = 'today you can only access the 3rd floor, not everything.'

Where Session Policies Are Applied

- ◆ `AssumeRole` — internal or cross-account role assumption
- ◆ `AssumeRoleWithWebIdentity` — OIDC / EKS IRSA
- ◆ `AssumeRoleWithSAML` — enterprise SSO

■ ■ Session policies are NOT used by regular IAM users making direct API calls. Only during STS-based role assumption.

Evaluation Logic

Final Permission = Role Policy $\cap$ Session Policy

Session policy can ONLY reduce permissions — never increase. Even if the session policy allows `ec2:*`, if the role policy doesn't allow it, EC2 is still denied.

Full CLI Example

Assuming Role with Session Policy via CLI

```
aws sts assume-role \ --role-arn arn:aws:iam::123456789012:role/AdminRole \
--role-session-name deploy-session \ --policy '{ "Version": "2012-10-17",
"Statement": [{ "Effect": "Allow", "Action": "s3:PutObject", "Resource":
"arn:aws:s3:::deploy-bucket/*" }] }' # AdminRole has: s3:*, ec2:*, iam:* # Session
policy: only s3:PutObject on deploy-bucket # Effective: ONLY s3:PutObject on
deploy-bucket
```


Real-World Use Cases

1 — Safer CI/CD Automation

Your CI/CD pipeline role has AdministratorAccess (needed for various deploy tasks). But for a specific deployment job, you assume the role with a session policy that allows ONLY s3:PutObject to the deployment bucket. If the pipeline is compromised, the attacker cannot destroy EC2, modify IAM, or touch any other resource.

2 — Temporary Read-Only Debugging Access

An engineer has a full-access role for their team account. For a production debugging session, they assume the role with a session policy restricting to read-only actions. Reduces blast radius if they accidentally run a destructive command.

3 — Multi-Tenant SaaS Isolation

A SaaS platform uses one IAM role for all customer data access. Each API request assumes the role with a session policy restricting access to that specific customer's S3 prefix (e.g., arn:aws:s3:::data-bucket/customer-123/*). Dynamic isolation without creating 100 separate roles.

Session Policy vs Permission Boundary — Side-by-Side

Feature	Permission Boundary	Session Policy
Applied to	IAM User or Role (the identity)	A specific temporary session
Lifetime	Permanent (until explicitly removed)	Temporary — expires with STS creds
Who sets it	Admin at identity creation/update time	The caller at assume-role time
Purpose	Limit maximum capability ever	Restrict current usage only
Grants permissions?	NO — only restricts	NO — only restricts
Analogy	"You can NEVER leave India"	"Today you can only go to Hyderabad"

Can a Session Policy Override a Permission Boundary?

■ NO. Session policy and Permission Boundary are independent filters. Final = Role Policy $\cap$ Boundary $\cap$ Session. All three must allow. A session policy cannot bypass the boundary, and vice versa.

Critical Edge Case — Session Policy Allows Something NOT in Role

Non-overlapping Session Policy Edge Case

```
Role Policy allows: s3:* Session Policy allows: ec2:* Result: ec2:* → DENIED (role
doesn't allow it; intersection = empty for EC2) s3:* → DENIED (session doesn't allow
it; intersection = empty for S3) Final: NO permissions at all – intersection of
non-overlapping sets is empty.
```

Security: Can a Stolen Session Policy Be Bypassed?

If an attacker steals temporary credentials that were issued with a session policy, can they bypass the restriction?

■ NO. The session policy is BAKED INTO the STS token at issuance time and is enforced server-side by AWS on every API call. The attacker cannot modify or remove it. Even if they use those creds, every request is evaluated against the embedded policy.

SCPs — Service Control Policies

AWS Organizations account-level guardrails

What SCPs Are

SCPs are AWS Organizations policies applied to OUs (Organizational Units) or individual accounts. They define the **MAXIMUM** permissions available to every IAM identity in the account — including the root user of the account.

Property	Detail
Applied at	AWS Organization root, OU, or individual account
Affects	ALL IAM users, roles, and root user in the account
Does it grant permissions?	NO — only filters what's already allowed by IAM
Overrides IAM?	YES — even AdministratorAccess is blocked if SCP denies
Cascades?	Yes — OU-level SCPs cascade to all child OUs and accounts

SCP + IAM Interaction

SCP = account-level guardrail. IAM = identity-level grants. Action is allowed **ONLY** if both SCP AND IAM policy allow it.

SCP Example — Prevent Bucket Deletion (Even by Admin)

```
# SCP on account – deny all S3 delete operations: { "Effect": "Deny", "Action":  
"s3:DeleteBucket", "Resource": "*" } # Result: Even account root cannot delete  
buckets. # This is the strongest guardrail in AWS.
```

■■ SCPs are the strongest denial mechanism in AWS. They override even AdministratorAccess. Use them for truly non-negotiable controls.

Resource-Based Policies

Policies attached to resources — enabling cross-account without role assumption

IAM Policy vs Resource Policy

Dimension	IAM (Identity-Based) Policy	Resource-Based Policy
Attached to	User, Group, Role	The AWS resource itself (S3, Lambda, SQS...)
Answers	"What can I do?"	"Who is allowed to access me?"
Cross-account	Requires AssumeRole	Can grant access directly — no role assumption needed
Principal field	Not present (principal = attached identity)	Required — specifies who gets access

Services That Support Resource-Based Policies

- ◆ **S3 Bucket Policy** — Cross-account access, public/private control, VPC endpoint restrictions
- ◆ **Lambda Resource Policy** — Allow other services (API Gateway, S3 events) to invoke the function
- ◆ **SNS Topic Policy** — Allow cross-account publishers or subscribers
- ◆ **SQS Queue Policy** — Allow cross-account producers/consumers or services to send/receive
- ◆ **KMS Key Policy** — Required — KMS key won't work without a key policy
- ◆ **ECR Repository Policy** — Cross-account pull access for container images
- ◆ **Secrets Manager** — Cross-account access to secrets

Real-World Issue: Public S3 Buckets

■ Public S3 buckets happen because resource policies are misconfigured — typically Principal: '*' with no conditions. Always audit resource policies alongside identity policies.

Fine-grained access control — where real IAM mastery begins

IAM Conditions — Overview

Conditions are optional elements in IAM policies that give you fine-grained control over WHEN a policy applies. They turn binary allow/deny into context-aware decisions.

Most Important Condition Keys

Condition Key	Use case	Example
aws:SourceIp	Allow access ONLY from specific IP ranges	Corporate office VPN only
aws:MultiFactorAuthPresent	Enforce MFA for sensitive actions	Require MFA to delete S3 objects
aws:RequestedRegion	Restrict actions to specific AWS regions	Only us-east-1, eu-west-1
aws:PrincipalTag	Tag-based access control (ABAC)	Department = Finance only
aws:ResourceTag	Access based on resource tags	Only EC2 tagged Environment=Prod
aws:CurrentTime	Time-based access windows	Business hours only
aws:SecureTransport	Require HTTPS (deny HTTP)	S3 bucket data in transit
s3:prefix	Restrict to specific S3 key prefixes	Customer-specific folder access

Condition Examples

Common Condition Examples

```
# Allow ONLY from corporate IP range: "Condition": { "IpAddress": { "aws:SourceIp":  
  "203.0.113.0/24" } } # Require MFA: "Condition": { "Bool": {  
  "aws:MultiFactorAuthPresent": "true" } } # Allow EC2 only if tagged  
Environment=Production: "Condition": { "StringEquals": {  
  "aws:ResourceTag/Environment": "Production" } }
```

ABAC — Attribute-Based Access Control

ABAC is a strategy where access decisions are based on TAGS rather than explicit role-to-resource mappings. Instead of creating 100 roles for 100 teams, you create one policy that grants access when tags on the principal MATCH tags on the resource.

ABAC Policy — Tag Matching

```
# Policy: allow access when principal tag matches resource tag { "Effect": "Allow",  
"Action": "s3:GetObject", "Resource": "*", "Condition": { "StringEquals": {  
"aws:PrincipalTag/Department": "${aws:ResourceTag/Department}" } } } # User tagged  
Department=Finance can ONLY access S3 buckets tagged Department=Finance # No need to  
create a Finance role AND a bucket policy per team — one policy scales to all.
```

Approach	Roles needed	Maintenance	Scalability
RBAC (Role-Based)	1 role per team/use-case	High — update policies per role	Poor — N teams = N roles
ABAC (Tag-Based)	1 policy for all teams	Low — just tag users and resources	Excellent — scales automatically

The Problem We're Solving

A Kubernetes pod needs AWS access (S3, DynamoDB, etc.). The naive solution causes serious security problems:

- ✗ **Hardcoding AWS keys inside the container** — keys leak, no rotation, blast radius = entire account
- ✗ **Using node-level IAM role** — ALL pods on that node get the SAME permissions, destroying least-privilege

The correct solution: each pod gets its OWN IAM role with MINIMUM required permissions, via temporary STS credentials that auto-rotate. Two mechanisms exist:

Method	How it works	Complexity	Status
EKS Pod Identity	Pod Identity Agent + association — agent intercepts IMDS requests	Low	NEW — AWS recommended 2024+
IRSA	OIDC token → STS AssumeRoleWithWebIdentity	Medium	Mature — widely used, still valid

PART 1 — EKS Pod Identity (Modern Way)

Step 1: Create the IAM Role with Pod Identity Trust Policy

Pod Identity Trust Policy

```
{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Principal": {  
  "Service": "pods.eks.amazonaws.com" }, "Action": ["sts:AssumeRole", "sts:TagSession"]  
}] } # NOTE: Principal is pods.eks.amazonaws.com — this is NEW, different from IRSA
```

Step 2: Attach a Permission Policy

Permission Policy Example

```
{ "Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::my-bucket/*"  
}
```

Step 3: Install the Pod Identity Agent Addon

Install Agent

```
aws eks create-addon \ --cluster-name my-cluster \ --addon-name  
eks-pod-identity-agent
```

Step 4: Create the Pod Identity Association (The Magic Step)

Link Service Account → IAM Role

```
aws eks create-pod-identity-association \ --cluster-name my-cluster \ --namespace  
default \ --service-account my-service-account \ --role-arn  
arn:aws:iam::123456789012:role/MyPodRole
```

Step 5: Create Kubernetes Service Account

Kubernetes Service Account

```
apiVersion: v1 kind: ServiceAccount metadata: name: my-service-account namespace:  
default # No annotations needed – Pod Identity uses the association instead
```

Step 6: Use in Pod

Pod Spec

```
apiVersion: v1 kind: Pod spec: serviceAccountName: my-service-account containers: -  
name: app image: amazonlinux command: ['sleep', '3600'] # Test inside pod: aws s3 ls  
– works WITHOUT any credentials in the pod
```

■ Pod Identity removes OIDC complexity. The agent intercepts the credential request and fetches STS creds on behalf of the pod. Zero secrets in the pod definition.

Common Pod Identity Mistakes

- ✗ Forgetting to install the eks-pod-identity-agent addon — pods won't get creds
- ✗ Wrong namespace or service account name in the association — mapping silently fails
- ✗ Trust policy uses the IRSA principal (oidc-provider) instead of pods.eks.amazonaws.com

PART 2 — IRSA (IAM Roles for Service Accounts — OIDC Method)

IRSA is the older approach that works by issuing OIDC JWT tokens to pods and having them call STS AssumeRoleWithWebIdentity directly.

IRSA Flow

- ◆ 1. EKS control plane issues a signed OIDC JWT token to the pod (projected service account token)
- ◆ 2. Pod's AWS SDK sends this token to STS: AssumeRoleWithWebIdentity

- ◆ 3. STS validates the JWT signature against the EKS cluster's registered OIDC provider
- ◆ 4. STS checks the role's trust policy — does it allow this OIDC provider + service account subject?
- ◆ 5. STS returns temporary credentials to the pod

IRSA Setup Steps

IRSA Step 1 & 2: Enable OIDC

```
# Step 1: Get cluster OIDC issuer URL
aws eks describe-cluster --name my-cluster \
  --query 'cluster.identity.oidc.issuer' --output text
# Step 2: Create OIDC provider in IAM (associate with cluster)
eksctl utils associate-iam-oidc-provider --cluster my-cluster --approve
```

IRSA Trust Policy

```
# Step 3: IAM Role trust policy for IRSA
{
  "Effect": "Allow",
  "Principal": {
    "Federated": {
      "arn:aws:iam::ACCOUNT:oidc-provider/oidc.eks.REGION.amazonaws.com/id/CLUSTER_ID"
    }
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "oidc.eks.REGION.amazonaws.com/id/CLUSTER_ID:sub": "system:serviceaccount:NAMESPACE:SERVICE_ACCOUNT_NAME"
    }
  }
}
```

IRSA Annotated Service Account

```
# Step 4: Annotate the Kubernetes Service Account
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-sa
  namespace: default
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/MyIRSARole
```

Pod Identity vs IRSA — Full Comparison

Feature	IRSA	Pod Identity
Mechanism	OIDC JWT token → STS	Agent intercepts IMDS
Setup complexity	High (OIDC provider, annotation, trust policy)	Low (addon + association)
Uses OIDC	Yes — required	No
Trust policy principal	Federated OIDC provider	pods.eks.amazonaws.com
Service account link	Annotation on SA	Association object in EKS
Security model	Token-based — strong identity validation	Agent-based — simpler, infra trust
Flexibility	Works with any OIDC-capable system	EKS-specific only
Status	Legacy-ish — still fully supported	AWS recommended for new workloads

Preventing Pod Compromise from Lateral Movement

If a pod is compromised, how do you prevent it from accessing another pod's IAM role?

- ✓ **Service Account Isolation** — Each microservice has its OWN Kubernetes service account with its OWN IAM role. Role A's trust policy only trusts service account A — service account B cannot assume Role A.
- ✓ **Network Policies** — Use Kubernetes NetworkPolicy to prevent lateral movement between pods. A compromised pod cannot reach the Pod Identity Agent of another namespace.
- ✓ **Namespace Isolation** — Separate high-sensitivity workloads into different namespaces with different RBAC. Pod Identity associations are namespace-scoped.
- ✓ **Least Privilege IAM** — Each pod's IAM role has ONLY the specific S3 bucket/DynamoDB table/KMS key it needs. Even if credentials are stolen, blast radius is minimal.
- ✓ **Short-Lived Credentials** — STS tokens expire (default 1h). An attacker has a limited window.

IAM Terminology Demystified

Identity · Entity · Principal · Resource — precise definitions for interviews and architecture

The Big Picture

AWS documentation uses these four terms loosely, causing widespread confusion. Here is the precise distinction:

A Principal uses an IAM Identity (backed by an IAM Entity) and attempts to access a Resource using policies.

IAM Entity

An IAM entity is any OBJECT you create inside IAM. It is the broadest term.

- ◆ IAM User — entity ✓, identity ✓ (can authenticate)
- ◆ IAM Group — entity ✓, identity ✗ (cannot authenticate or make API calls directly)
- ◆ IAM Role — entity ✓, identity ✓
- ◆ IAM Policy — entity ✓, identity ✗ (just a permission document)

■ ■ Think: 'Everything that exists as a named object inside IAM is an entity.'

IAM Identity

An IAM identity is an entity that AWS can AUTHENTICATE — meaning it can receive and use credentials.

- ◆ IAM User — has a password and/or access keys
- ◆ IAM Role — can be assumed (generates STS creds)
- ◆ Federated Identity — authenticated via SAML or OIDC, mapped to a role

■ ■ Groups and Policies are entities but NOT identities — they cannot authenticate to AWS.

Principal

A principal is the RUNTIME ACTOR making a request to AWS. It is broader than IAM identity because it includes non-IAM actors.

- ◆ IAM User (arn:aws:iam::123:user/tharun)

- ◆ IAM Role Session (arn:aws:sts::123:assumed-role/MyRole/session-name)
- ◆ AWS Service (ec2.amazonaws.com, lambda.amazonaws.com)
- ◆ Another AWS Account (arn:aws:iam::456789:root)
- ◆ Federated SSO User
- ◆ Anonymous public (Principal: '*' — use carefully!)

■ ■ When a role is ASSUMED, the principal is the assumed-role SESSION, not the role definition itself. ARN: arn:aws:sts::123:assumed-role/RoleName/SessionName

Resource

A resource is the AWS object being ACCESSED by the principal.

- ◆ S3: arn:aws:s3::my-bucket or arn:aws:s3::my-bucket/*
- ◆ EC2: arn:aws:ec2:us-east-1:123:instance/i-0abc123
- ◆ DynamoDB: arn:aws:dynamodb:us-east-1:123:table/orders
- ◆ Lambda: arn:aws:lambda:us-east-1:123:function:process
- ◆ KMS: arn:aws:kms:us-east-1:123:key/key-id

All Four Terms — EKS Real-World Example

Term	In the EKS Pod → S3 scenario
IAM Entity	Role: EKS-Pod-S3-Role and Policy: ReadS3Policy — objects in IAM
IAM Identity	EKS-Pod-S3-Role — the assumable role
Principal	arn:aws:sts::123:assumed-role/EKS-Pod-S3-Role/pod-session — the active session
Resource	arn:aws:s3::app-data/* — the S3 objects being read

Interview-Ready Precision

If asked...	Say...
Difference between Role and Principal?	Role is an IAM identity DEFINITION (a named object in IAM). Principal is the RUNTIME ACTOR — typically an assumed-role session — making the API request.
Is IAM Group a Principal?	Usually no — groups cannot make API calls directly. Users in groups can, using their user identity.

Is a Policy an Identity?	No — a policy is attached permissions, not an identity. It defines WHAT, not WHO.
Who is the principal in Lambda→Lambda?	The EXECUTION ROLE session of the invoking Lambda — i.e., <code>arn:aws:sts::...assumed-role/LambdaExecRole/FunctionName</code> .

What to never do — and how to fix it

Top IAM Anti-Patterns

1. Using root account for anything

Root has unrestricted access to everything. It cannot be restricted by SCPs. Create an IAM admin user/role for daily use. Enable MFA on root and lock away the credentials.

2. Hardcoding credentials in code

Credentials in source code leak via git history, logs, crash dumps. Use IAM roles for EC2/Lambda, Pod Identity/IRSA for EKS, Secrets Manager for external services.

3. Over-permissive wildcards (Action: '*' or Resource: '*')

Violates least privilege. If compromised, attacker gets everything. Scope to specific actions and resource ARNs.

4. No key rotation

Long-lived access keys are the biggest breach vector. Use temporary STS credentials wherever possible. Rotate static keys every 90 days.

5. Sharing IAM users across people/services

Destroys audit trail — you cannot tell WHO did WHAT. 1 user / 1 person; use roles for services.

6. Ignoring CloudTrail

IAM without audit logging is blind. Enable CloudTrail in ALL regions. Alert on iam:CreateUser, iam:AttachRolePolicy, iam:CreateAccessKey.

7. Not using MFA for sensitive operations

Critical: iam>DeleteRole, ec2:TerminateInstances should require MFA via aws:MultiFactorAuthPresent condition.

8. Trusting Principal: '*' without conditions

In resource policies, '*' principal without strict conditions is effectively public access. Always add conditions (aws:SourceVpc, aws:SourceAccount, etc.).

9. Not using Permission Boundaries when delegating IAM

Allowing devs to create roles without boundaries enables privilege escalation — dev creates a role with more power than themselves.

10. Debugging IAM by only checking identity policy

ALWAYS check all 5 layers: Identity policy, SCP, Boundary, Resource policy, Session policy. One layer can silently deny.

AccessDenied Master Checklist

AccessDenied Debugging Checklist

WHEN YOU SEE AccessDenied – check in this order: [] 1. Check CloudTrail for the exact denied action and resource ARN [] 2. Is there an explicit Deny in any attached IAM policy? [] 3. Is there an SCP in the AWS Org blocking this action? [] 4. Is there a Permission Boundary limiting the role? [] 5. Was the role assumed with a Session Policy that restricts this? [] 6. Does the resource-based policy (S3, KMS, etc.) deny or not include this principal? [] 7. Is the Trust Policy correct? (Can this entity even assume the role?) [] 8. Is the resource ARN correct? (Typos, wrong region, wrong account ID) [] 9. Is the action region-restricted by a condition? [] 10. Is VPC endpoint policy blocking the traffic?

Architect-Level IAM Strategy

When designing IAM for a real production system:

- ✓ Use SCPs for org-wide non-negotiables (deny root access, deny specific regions, deny disabling CloudTrail)
- ✓ Use Permission Boundaries when delegating IAM creation to dev teams
- ✓ Use Session Policies for automation/CI-CD pipelines to scope down the blast radius
- ✓ Use ABAC (tag-based) for scalable access control across many teams and resources
- ✓ Use Pod Identity (or IRSA) for EKS workloads — never node-level roles or static creds
- ✓ Use Cross-Account Roles with External ID for all third-party integrations
- ✓ Use Resource Policies for cross-account resource access (avoids role assumption complexity)
- ✓ Enable CloudTrail + Config + GuardDuty — IAM without audit is blind
- ✓ Review IAM Access Analyzer monthly — it flags overly permissive policies automatically